

AI Engineering Design Brief

Retail Analytics Assistant — AI Solution Design Round

Candidate: Alakh Babbar

Scope: Design of an AI-assisted analytical interface for an FMCG retail dataset, covering promotional effectiveness, sales, inventory, and regional performance.

Section A: Problem Decomposition

A.1 — User Questions the Assistant Should Support

Business users interacting with this assistant are not analysts. They ask questions the way they would ask a colleague, not the way they would write SQL. The design groups the expected questions into five categories, based on the analytical fields present in the dataset (region, state, city, product, category, brand, promotion, inventory, revenue, and rating):

- Revenue and sales questions — e.g. "Which region generated the highest revenue last quarter?", "How did Soft Drinks perform compared to Snacks?"
- Promotion and campaign effectiveness — e.g. "Did last month's campaign improve sales in the South region?", "Which promotion type performed best?"
- Inventory movement — e.g. "Which categories saw inventory reduction during promotions?", "Are any products running low on stock?"
- Regional and comparative performance — e.g. "Compare North and South revenue for Q2", "Which state has the highest average customer rating?"
- Definitional / business-concept questions — e.g. "What is Promotion Lift?", "How is Baseline Revenue calculated?" — these do not require touching the dataset at all.

A meaningful share of realistic questions are hybrid: they need a number from the data and an interpretation grounded in a business definition in the same answer. "Did the campaign improve sales in the South region?" is a good example — it requires a SQL aggregation (revenue during vs. before the campaign) and the Promotion Lift definition to interpret the result correctly. The design treats this as the default case rather than an edge case.

A.2 — Structured Analytical Outputs

The assistant is expected to produce two things for most questions, not one:

- A grounded numeric result — computed by SQL execution against the analytical table, never generated freeform by the language model.
- A short natural-language explanation — synthesized from the numeric result plus, where relevant, a retrieved business definition (e.g. explaining what a lift percentage means in context).

For comparative or trend questions, the underlying result is a small table (region-by-revenue, week-by-units, etc.); the assistant summarizes it rather than dumping the raw table, though the raw table is available if the user wants it.

Section B: AI Solution Approach

B.1 — Methods Used

The solution deliberately avoids model training. Three techniques do the actual work:

- Prompt engineering — structured, schema-aware prompts drive both SQL generation and the final response synthesis, with explicit business rules injected rather than assumed.
- SQL generation (text-to-SQL) — the language model translates a natural-language analytical task into a single SQL query against the SQLite database, using a schema that is inspected at runtime rather than hardcoded.
- Retrieval-augmented generation (RAG) — a separate knowledge base (knowledge.md) holding KPI formulas, aggregation rules, and business terminology is embedded and searched via cosine similarity so the model doesn't have to "remember" definitions.

B.2 — Why This Approach Is Suitable

Retail analytics questions are ultimately about numbers, and numbers derived from an LLM's own generation are not trustworthy — a model can produce a plausible-sounding revenue figure that is simply wrong. Routing every numeric claim through actual SQL execution against the source table removes that failure mode for the data itself. Separately, definitions like Promotion Lift or Baseline Revenue are static business rules, not facts that change per question — a small retrieval corpus is a better fit for these than either hardcoding them into every prompt or trusting the model's parametric knowledge, which has no way of knowing this business's specific formulas. Splitting the two concerns (SQL for computation, RAG for definitions) keeps each component doing one job well instead of asking a single LLM call to both reason about numbers and recall business rules simultaneously.

B.3 — Agentic Construct

The word "agent" is used narrowly here. This is not an autonomous system that sets its own goals or revises its plan after seeing intermediate results. The agentic construct is a bounded planner–executor pipeline: a planning step decides, per question, which of two tools are needed (SQL, retrieval, or both), a fixed set of tools execute deterministically, and a final synthesis step composes the answer. The planner never answers the question itself — it only decides what evidence to gather. This is closer to a structured pipeline with one decision point than to a general-purpose autonomous agent, and that distinction matters for how its limitations are reasoned about (Section D).

Section C: Data Interaction Strategy

C.1 — Accessing Analytical Datasets

The analytical dataset, sales_data.csv, is loaded into a SQLite database at the start of a session rather than being queried as a flat file. This gives the assistant a real query engine instead of ad-hoc Pandas filtering logic. Rather than hardcoding column names or table structure into the prompts, the notebook inspects the resulting SQLite database — its tables, columns, inferred types, and a handful of sample values — and injects that schema description directly into the SQL generation prompt. This keeps SQL generation correct if the dataset's structure changes without requiring prompt edits, and it avoids a common failure mode where a model invents a plausible but nonexistent column name.

C.2 — Performing Aggregations

Aggregation logic is not left to the model's judgment at answer time. The knowledge base encodes explicit rules — revenue and units are summed, customer rating is averaged rather than summed, and inventory columns are treated as snapshots that must never be summed across weeks — and these rules are surfaced to the SQL generation step alongside the schema. The generated SQL executes directly against SQLite, and the result set is converted to a Pandas DataFrame and then a compact text table before being handed to the response generation step, so the final natural-language answer is built on top of an already-computed, already-correct number rather than being asked to compute or restate it from memory.

Section D: Limitations & Risks

D.1 — Incorrect Query Generation

The SQL generation step is a single LLM call producing a single query, with no verification layer checking that the query is logically correct before it runs, and no retry loop if it fails. An invalid or subtly wrong query — for example, one that joins on the wrong grain, or misapplies an aggregation the knowledge base didn't cover — will either error out or silently return a misleading number.

Mitigation: grounding SQL generation in the real, inspected schema (Section C.1), explicit business/aggregation rules injected into the prompt, and a near-zero temperature to reduce variance in query structure between runs. This reduces the frequency of errors; it does not eliminate them, and the current implementation has no automatic correction step.

D.2 — Hallucinated Responses

Because the final response generator is itself an LLM call, there is always some risk it states a number that doesn't match its inputs, or blends a retrieved definition with an invented detail.

Mitigation: numeric claims are constrained to come from the SQL result table, and conceptual claims are constrained to come from retrieved knowledge-base chunks, with the prompt structured to keep those two sources separate rather than letting the model reason freely. Temperature is kept near zero for the same reason.

D.3 — Data Misinterpretation

The planner's job is to translate a natural-language question into a SQL task description and a retrieval task description in a single pass, with no opportunity to revise that plan after seeing the SQL result. A question that is ambiguous about time range, region grouping, or which metric "sales" refers to (revenue vs. units) can be mis-specified at the planning stage and never corrected downstream.

Mitigation: the knowledge base defines defaults for common ambiguities (e.g. "sales" defaults to revenue unless the question says otherwise), which reduces but does not remove this risk, since the planner has to actually retrieve and apply that convention correctly on every run.

Section E: Design Trade-Off

Accuracy Over Latency

The pipeline is intentionally multi-stage: plan, generate SQL, execute SQL, retrieve knowledge, then synthesize a response. That is three to four sequential LLM calls for a single user question, where a simpler design could have asked one model to read the question and the raw data and answer directly in one call.

The one-call approach was rejected because it reintroduces the exact problem this design is trying to avoid: an LLM asserting a number instead of computing it. Splitting the pipeline into discrete stages, each with a narrow and checkable job, costs latency — a user waits noticeably longer than they would for a single completion — but it means every numeric claim in the final answer can be traced back to an actual SQL result rather than a generation. For an assistant whose entire value proposition is replacing manual analyst work, a slower but verifiably correct number was judged more valuable than a fast but unverifiable one.
